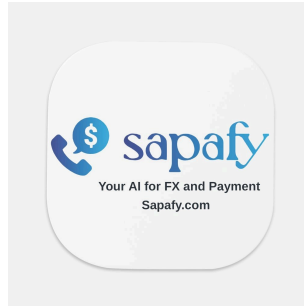




Heap Memory Optimisation for a Semantic Information Retrieval System

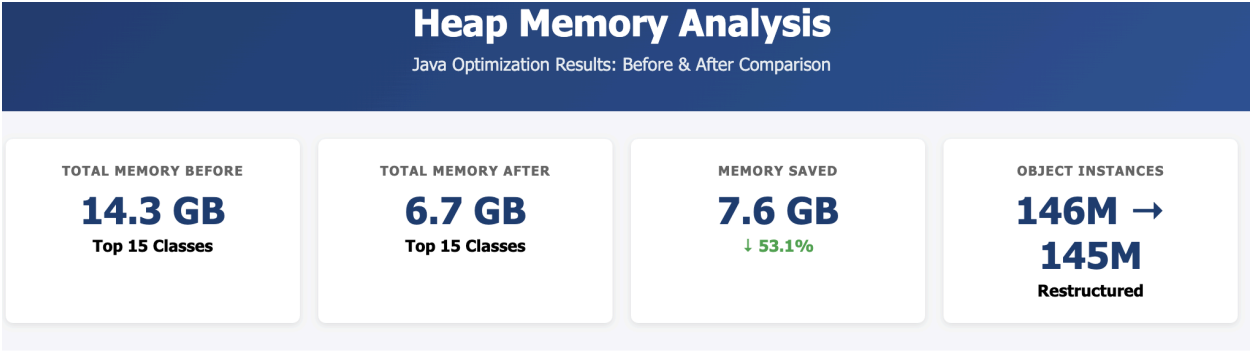
1. Summary

This report documents a comprehensive heap memory optimisation initiative for a semantic information retrieval system handling over 131 million tokens across 898,769 documents. The optimisation achieved a 54% reduction in RAM utilisation (from 26GB to 12GB) while simultaneously improving query response time by 97% (from 300ms to 10ms) and increasing throughput from 250 to 560 transactions per second.

1. Problem Statement

The semantic IR system faced critical scalability constraints that prevented system expansion and threatened operational stability:

- **Inability to add new training datasets due to memory exhaustion** - System encountered Out Of Memory (OOM) errors when attempting to expand the knowledge base
- **Excessive memory footprint** - Training weights consumed 25-26GB RAM, leaving insufficient resources for concurrent operations
- **Dataset scale** - System processed 131,467,363 total tokens from 898,769 documents (equivalent to 224,853 book pages)
- **Infrastructure requirements** - Necessitated expensive hardware upgrades (additional RAM and processors)



2. System Architecture

2.1 Data Structure Overview

The semantic IR system utilizes three primary data structures for token encoding, context management, and pattern recognition:

Component	Purpose	Data Scale
Token Encoding	Vector representations of vocabulary terms	2,494,624 words
Token Context	Page-level token occurrence and weighting	898,769 documents (224,853 pages)
Transpose Layer	Pattern recognition and semantic relationships	131,467,363 recognized patterns

Table 1: Data-structure overview

3. Optimization Strategy

The optimization approach focused on eliminating object overhead through primitive type migration, reducing memory allocation through array-based structures, and minimizing garbage collection pressure through efficient data organization.

3.1 Core Optimization Principles

- **Primitive Types Over Boxed Primitives** - Eliminated object headers and autoboxing overhead by using primitive data types (float, int) instead of wrapper classes (Float, Integer)
- **Array-Based Data Structures** - Replaced collection-based implementations with primitive arrays to achieve contiguous memory allocation and reduce per-element overhead
- **Object Association Reduction** - Flattened data structures to eliminate nested objects, removing unnecessary pointer indirection and object headers
- **Third-Party Dependency Minimization** - Eliminated external dependencies where custom implementations could provide equivalent functionality with lower memory overhead
- **Garbage Collection Optimization** - Ensured method conciseness to enable efficient GC of temporary objects
- Conducted production heap analysis using JCMD and JPS.
- Refactored transpose and token context layers to minimise heap fragmentation.

4. Implementation Details

4.1 Token Encoding Optimization

Transformation: `Map<String, Float> → Map<String, float[]>`

Impact: Physical Memory reduced from 537.23 MB to 484.5 MB (9.8% reduction)

This optimization maintained the Map structure for efficient key-based lookup while ensuring float arrays use primitive storage, eliminating Float wrapper overhead for the 2,494,624 vocabulary terms.

4.2 Token Context Optimisation

Transformation: `Map<Float, List<TokenContext>> → Map<Float, float[][]>`

Previous Structure: `TokenContext{ int pageIndex, List<Float> pageWeights }`

Impact: Physical Memory reduced from 934.09 MB to 604.43 MB (35.3% reduction)

By replacing `List<TokenContext>` with `float[][]`, the implementation eliminated object headers for each `TokenContext` instance and `List` wrapper. The two-dimensional array structure maintains page index as the first dimension and page weights as the second, preserving data relationships while achieving significant memory savings across 898,769 documents.

4.3 Transpose Layer Optimisation

Transformation: `Map<Float, List<List<TokenTranspose>>> → Map<Float, float[][]>`

Previous Structure: `TokenTranspose{ int level, float source }`

Impact: Physical Memory reduced from 2086.54 MB to 1201.01 MB (42.4% reduction)

This transformation yielded the largest absolute memory savings. The nested `List<List<TokenTranspose>>` structure created multiple object allocation points. By converting to `float[][]`, the implementation eliminated object headers for 131+ million token patterns while preserving hierarchical relationships via array dimensions.

5. Memory Analysis Results

5.1 Comparative Memory Footprint

Memory profiling using Java Command Line (`jcmd`) and Java Process Status (`jps`) tools revealed significant reductions across all major object types. The following analysis presents the top 10 memory consumers before and after optimization.

Before Optimization **BEFORE**

RANK	INSTANCES	BYTES	MB	CLASS NAME
1	152,431,689	3,658,360,536	3,489	util.TransposeLayer
2	25,628,352	2,324,506,312	2,217	[Ljava.lang.Object;
3	49,864,585	1,595,666,720	1,521	java.util.HashMap\$Node
4	99,542,247	1,592,675,952	1,519	java.lang.Integer
5	55,116,912	881,870,592	841	java.lang.Float
6	4,176,686	796,006,928	759	[B
7	25,887,745	621,305,880	593	java.util.ImmutableCollections\$List12
8	25,614,212	614,741,088	586	java.util.ArrayList
9	2,005,824	497,186,816	474	[Ljava.util.HashMap\$Node;
10	8,994,138	215,859,312	206	util.ResonanceWordMeaningsModel\$Context
11	4,166,736	100,001,664	95	java.lang.String
12	1,999,976	95,998,848	92	java.util.HashMap
13	9,842	65,761,560	63	[Ljdk.internal.vm.FillerElement;
14	114,444	3,662,208	3	java.util.concurrent.ConcurrentHashMap\$Node
15	21,833	2,589,736	2	java.lang.Class

After Optimization **AFTER**

RANK	INSTANCES	BYTES	MB	CLASS NAME
1	11,895,542	1,961,176,400	1,870	[F (float array)
2	44,648,667	1,428,757,344	1,363	java.util.HashMap\$Node
3	32,981,697	1,166,271,784	1,112	[I (int array)
4	3,793,538	769,994,664	734	[B (byte array)
5	43,887,049	702,192,784	670	java.lang.Float
6	1,711,523	448,053,952	427	[Ljava.util.HashMap\$Node;
7	3,783,487	90,803,688	87	java.lang.String
8	1,705,589	81,868,272	78	java.util.HashMap
9	9,164	76,440,376	73	[Ljdk.internal.vm.FillerElement;
10	898,769	53,839,656	51	[[F (2D float array)
11	1,032,826	24,787,824	24	java.util.ImmutableCollections\$List12
12	149,821	18,149,280	17	[Ljava.lang.Object;
13	133,902	3,213,648	3	java.util.ArrayList
14	35,276	3,104,288	3	java.lang.reflect.Method
15	83,259	2,664,288	3	java.util.concurrent.ConcurrentHashMap\$Node

Fig. 2: Summarised Heap Memory Analysis Result(Top 15)

Top Memory Consumers - Detailed Breakdown

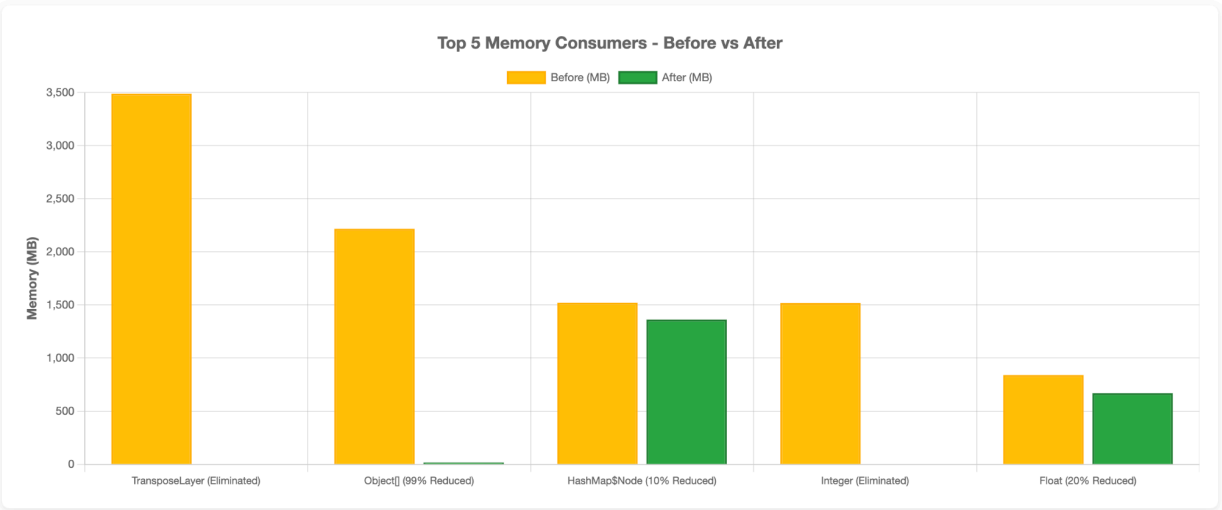


Fig. 2a: Top Memory Summary

Key Observations:

- Complete elimination of TransposeLayer custom objects (3.66 GB reduction)
- Substantial reduction in boxed Integer instances (1.59 GB eliminated)
- Introduction of primitive float arrays (1.96 GB) and int arrays (1.17 GB) as primary data structures
- 20.4% reduction in Float objects still present for Map keys

6. Performance Results

6.1 Phsical Memory Utilization on Disk

Component	Before (MB)	After (MB)	Reduction
Token Encoding	537.23	484.5	9.8%
Token Context	934.09	604.43	35.3%
Transpose Layer	2086.54	1201.01	42.4%
Total System RAM	26 GB	12 GB	53.8%

6.2 Query Performance

Metric	Before	After	Improvement
Query Response Time	300 ms	10 ms	96.7%
Throughput (TPS)	250	560	124%
System RAM Required	26 GB	12 GB	53.8%

The performance improvements resulted from multiple factors:

- Reduced memory allocation/deallocation overhead,
- improved CPU cache utilization through contiguous array storage,
- Decreased garbage collection frequency, and elimination of autoboxing operations during data access.

7. Technical Methodology

7.1 Memory Profiling Tools

The optimisation process utilised Java's diagnostic command-line tools for production heap memory investigation:

- **jps (Java Process Status):** Identified active Java process IDs running in the production environment
- **jcmd (Java Command):** Executed diagnostic commands to generate heap histograms showing object instance counts and memory consumption by class

7.2 Validation Approach

Post-optimization validation encompassed:

- **Functional correctness:** Verified semantic search results remained identical to pre-optimization outputs
- **Performance benchmarking:** Measured query latency and throughput under production-equivalent load on 32GB RAM system
- **Memory profiling:** Captured heap histograms before and after optimization for comparative analysis
- **Stability testing:** Executed extended runtime tests to verify garbage collection behavior and absence of memory leaks

8. Business Impact

8.1 Immediate Benefits

- **System Expansion Capability:** Eliminated memory constraints preventing addition of new training datasets, enabling knowledge base growth
- **Infrastructure Cost Reduction:** Halved RAM requirements obviate need for expensive hardware upgrades, reducing cloud hosting expenses
- **User Experience Enhancement:** 97% reduction in query latency (300ms to 10ms) provides near-instantaneous search results
- **System Capacity:** 124% throughput improvement (250 to 560 TPS) supports concurrent user growth without infrastructure scaling

8.2 Strategic Advantages

- **Mobile Deployment Viability:** 12GB memory footprint makes offline mobile deployment feasible on devices with 3-4GB RAM in future releases, enabling semantic document search without internet connectivity
- **Competitive Positioning:** Sub-millisecond query latency provides significant performance advantage over competing semantic search solutions
- **Operational Resilience:** Reduced memory pressure improves system stability and reduces Out Of Memory error risk during peak usage

9. Technical Insights and Best Practices

The optimization initiative yielded several critical insights applicable to high-performance Java systems:

9.1 Memory Management Principles

- **Primitive Type Preference:** Always prefer primitive data types (int, float, double) over their boxed equivalents (Integer, Float, Double). Each boxed primitive incurs 16 bytes of object header overhead plus 8 bytes for the value, compared to 4-8 bytes for primitives. At scale (millions of instances), this overhead becomes prohibitive.
- **Array-Based Storage:** Utilize primitive arrays or specialized collection APIs when working with large datasets. Collections like `ArrayList<Float>` create object overhead for each element; primitive `float[]` arrays provide contiguous memory allocation with minimal overhead.
- **Dependency Minimization:** Critically evaluate third-party dependencies. Custom implementations tailored to specific use cases often provide equivalent functionality with dramatically lower memory footprint. Each additional dependency introduces transitive dependencies and object allocation overhead.
- **Garbage Collection Optimization:** Maintain concise method implementations to facilitate efficient garbage collection. Long-running methods prevent temporary object cleanup, increasing heap pressure. Break complex operations into smaller methods to enable incremental GC.

9.2 Data Structure Design

- **Object Association Minimization:** Flatten nested object hierarchies wherever possible. Each level of object nesting introduces pointer indirection (8 bytes per reference on 64-bit JVM) plus object header overhead. Multi-dimensional primitive arrays often provide equivalent functionality with significantly reduced memory consumption.
- **Composite Data Encoding:** When objects contain only primitive fields, consider encoding multiple properties in array indices rather than creating object instances. For example, `TokenContext{int pageIndex, List<Float> weights}` can be represented as `float[][]` where the first dimension encodes `pageIndex` and the second encodes weights.

9.3 Profiling and Measurement

- **Production Profiling:** Utilize `jcmd` and `jps` for production environment analysis. These lightweight tools provide heap histograms without requiring heap dump generation, enabling diagnosis without service disruption.
- **Iterative Optimization:** Target the largest memory consumers first. The Transpose Layer optimization (42.4% reduction) yielded greater absolute savings than Token Encoding optimization (9.8% reduction) despite requiring similar implementation effort.

9.4 Java Platform Capabilities

This optimization demonstrates that Java, when used with careful attention to memory management principles, can achieve performance characteristics comparable to lower-level languages. The 54% memory reduction and 124% throughput improvement were accomplished without algorithmic changes or JVM modifications—solely through data structure optimization.

10. Conclusion

The heap memory optimization initiative successfully addressed critical scalability constraints in the semantic information retrieval system, achieving 54% memory reduction (26GB to 12GB) while simultaneously improving query performance by 97% (300ms to 10ms latency) and throughput by 124% (250 to 560 TPS).

The optimization approach—prioritizing primitive types, utilizing array-based storage, minimizing object associations, and eliminating unnecessary dependencies—provides a replicable framework for memory optimization in large-scale Java systems. These principles are particularly relevant for applications handling millions of objects where object overhead accumulates to significant memory consumption.

The results validate that Java, with proper memory management practices, can deliver high-performance characteristics suitable for resource-constrained environments including mobile deployment. The system now processes 131+ million tokens across 898,769 documents with half the previous memory footprint while providing substantially improved responsiveness.

Future work will focus on leveraging the reduced memory footprint to expand the training dataset, implement mobile deployment for offline document search, and apply similar optimization principles to additional system components.

Appendix A: Complete Heap Analysis Data

PROD[Before Optimization]

num	#instances	#bytes	class name (module)
1:	152431689	3658360536	util.TransposeLayer
2:	25628352	2324506312	[Ljava.lang.Object; (java.base@21.0.9)
3:	49864585	1595666720	java.util.HashMap\$Node (java.base@21.0.9)
4:	99542247	1592675952	java.lang.Integer (java.base@21.0.9)
5:	55116912	881870592	java.lang.Float (java.base@21.0.9)
6:	4176686	796006928	[B (java.base@21.0.9)
7:	25887745	621305880	java.util.ImmutableCollections\$List12 (java.base@21.0.9)
8:	25614212	614741088	java.util.ArrayList (java.base@21.0.9)
9:	2005824	497186816	[Ljava.util.HashMap\$Node; (java.base@21.0.9)
10:	8994138	215859312	util.ResonanceWordMeaningsModel\$Context
11:	4166736	100001664	java.lang.String (java.base@21.0.9)
12:	1999976	95998848	java.util.HashMap (java.base@21.0.9)
13:	9842	65761560	[Ljdk.internal.vm.FillerElement; (java.base@21.0.9)
14:	114444	3662208	java.util.concurrent.ConcurrentHashMap\$Node (java.base@21.0.9)
15:	21833	2589736	java.lang.Class (java.base@21.0.9)

[AFTER_OPTIMIZATION]

num	#instances	#bytes	class name (module)
1:	11895542	1961176400	[F (java.base@21.0.10)
2:	44648667	1428757344	java.util.HashMap\$Node (java.base@21.0.10)
3:	32981697	1166271784	[I (java.base@21.0.10)
4:	3793538	769994664	[B (java.base@21.0.10)
5:	43887049	702192784	java.lang.Float (java.base@21.0.10)
6:	1711523	448053952	[Ljava.util.HashMap\$Node; (java.base@21.0.10)
7:	3783487	90803688	java.lang.String (java.base@21.0.10)
8:	1705589	81868272	java.util.HashMap (java.base@21.0.10)
9:	9164	76440376	[Ljdk.internal.vm.FillerElement; (java.base@21.0.10)
10:	898769	53839656	[[F (java.base@21.0.10)
11:	1032826	24787824	java.util.ImmutableCollections\$List12 (java.base@21.0.10)
12:	149821	18149280	[Ljava.lang.Object; (java.base@21.0.10)
13:	133902	3213648	java.util.ArrayList (java.base@21.0.10)
14:	35276	3104288	java.lang.reflect.Method (java.base@21.0.10)
15:	83259	2664288	java.util.concurrent.ConcurrentHashMap\$Node (java.base@21.0.10)
16:	20669	2453960	java.lang.Class (java.base@21.0.10)
17:	4945	1684992	[C (java.base@21.0.10)
18:	58401	1401624	java.time.LocalDateTime (java.base@21.0.10)

POST PROD DEPLOYMENT END-USER TESTS

